

 ESPE UNIVERSIDAD DE LAS FUERZAS ARMADAS INNOVACIÓN PARA LA EXCELENCIA TEMA: Taller 3: Plan de Pruebas	Departamento: Departamento de Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y Diseño de Software NRC: 30723 Fecha: 09/07/2026
	Integrantes Grupo 3: • Ayuquina Danny. • Cañarte Saray. • Villagómez Doménica.

PROYECTO: FICITAS

Plan de Pruebas

1. Identificador del Plan de Pruebas: PP-FICITAS-2026-01 - Versión 1.0

2. Introducción y Alcance

Este documento define la estrategia, los casos de pruebas unitarias y las responsabilidades para validar los 7 requisitos funcionales del backlog del sistema FICITAS (gestión de un centro de fisioterapia), correspondientes a los Sprints 0 al 4: inicio de sesión, recuperación de contraseña, gestión de perfiles, gestión de pacientes, gestión de citas, gestión de planificación y notificaciones.

El sistema está desarrollado en JavaScript / Node.js, por lo que el plan contempla pruebas unitarias compatibles con ese stack (Jest, Supertest).

Fuera de alcance: pruebas de carga masiva (>conurrencia básica), pruebas de accesibilidad y auditoría de seguridad ofensiva (pentesting), ya que no están contempladas en los requisitos actuales del backlog.

3. Requisitos a probar

Módulo	Requisito	Build/Sprint
Autenticación	REQ000	Sprint 0
Recuperación de contraseña	REQ001	Sprint 0
Perfiles y roles	REQ005	Sprint 1
Pacientes	REQ004	Sprint 2
Citas	REQ003	Sprint 3
Planificación (horarios/consultorios)	REQ006	Sprint 4
Notificaciones	REQ002	Sprint 4

4. Herramientas y Configuración

- Jest como framework principal (test runner + assertions + mocks).
- `jest.fn()` / `jest.spyOn()` para simular repositorios (guardar, listar, actualizar, obtenerPorId, login).
- `jest.useFakeTimers().setSystemTime(...)` para controlar `Date.now()` en pruebas de fechas límite (cita en el pasado, duración, fecha de nacimiento).
- `jsdom` (entorno por defecto de Jest) para `NotificacionObserver`, ya que usa `document.getElementById` — o bien mockear `global.document/global.alert` directamente.
- `jest.mock('./datos/HorarioRepo.js')` (y equivalentes) para aislar `ServicioPlanificacion`, que no permite inyectar los repos por constructor.

5. Criterios de Aceptación

- Cobertura mínima 80% líneas / 70% ramas en las clases de servicio y decoradores.
- Toda regla de negocio con condición límite (fechas, cupos, duraciones, solapamientos) debe tener al menos un caso en el límite exacto y uno justo fuera del límite.
- Cero llamadas reales a repos, DOM o red — todo mockeado.

6. Casos de Prueba por Requisito

REQ000 — Inicio de Sesión

ID	Método	Entrada	Salida esperada	Técnica
UT-000-1	<code>ValidacionDecorador.iniciarSesion</code>	correo con ' <code><script></code> ' o ' <code>javascript:</code> '	{ <code>exito:false</code> , <code>mensaje:'La entrada contiene contenido no permitido'</code> }	Inyección/seguridad
UT-000-2	<code>ValidacionDecorador.iniciarSesion</code>	correo sin '@' o formato inválido	{ <code>exito:false</code> , <code>mensaje:'El correo electrónico no es válido'</code> }	Clase inválida
UT-000-3	<code>ValidacionDecorador.iniciarSesion</code>	<code>pass = ''</code> (vacío)	{ <code>exito:false</code> , <code>mensaje:'La contraseña es requerida'</code> }	Valor límite
UT-000-4	<code>ValidacionDecorador.iniciarSesion</code>	correo y pass válidos	delega en <code>servicioBase.iniciarSesion(correo, pass)</code>	Delegación (spy)
UT-000-5	<code>ServicioAutenticacion.iniciarSesion</code>	<code>usuarioRepo.login</code> devuelve { <code>exito:false</code> }	{ <code>exito:false</code> , <code>mensaje: <el de repo o 'Credenciales inválidas'></code> }	Clase inválida
UT-000-6	<code>ServicioAutenticacion.iniciarSesion</code>	<code>usuarioRepo.login</code> devuelve { <code>exito:true</code> , <code>usuario</code> , <code>perfil</code> }	<code>gestor.notificar</code> llamado 1 vez con { <code>tipo:'LOGIN'</code> , <code>datos:{correo}</code> }; retorna { <code>exito:true</code> , <code>usuario</code> , <code>perfil</code> }	Mock/spy

UT-000-7	UsuarioRepo.login	mock de fetch responde {exito:false, mensaje:'Cuenta bloqueada'}	ServicioAutenticacion.iniciar Sesion propaga el mensaje del backend sin alterarlo	Integración simulada (mock de fetch)
----------	-------------------	--	---	--------------------------------------

REQ001 — Recuperación de Contraseña

ID	Función (a confirmar)	Entrada	Salida esperada	Técnica
UT-001-1	generarCodigoVerificacion (nombre supuesto)	—	código de 6 dígitos	Diseño — pendiente código
UT-001-2	validarCodigo (nombre supuesto)	código vigente vs. expirado	true / false	Diseño — pendiente código
UT-001-3	puedeRecuperar (nombre supuesto)	fecha última recuperación a 89 y 91 días	false / true (límite 3 meses)	Diseño — pendiente código

REQ005 — Gestionar Perfiles

ID	Método	Entrada	Salida esperada	Técnica
UT-005-1	ServicioPerfiles.registrarPerfil	nombre, descripción, permisos válidos	perfilRepo.guardar llamado 1 vez; gestor.notificar con {tipo:'PERFIL_REGISTRADO'}; retorna {exito:true}	Mock/spy
UT-005-2	ServicioPerfiles.modificarPerfil	id inexistente (perfilRepo.actualizar devuelve null/undefined)	{exito:false, mensaje:'Perfil no encontrado'}	Clase inválida
UT-005-3	ServicioPerfiles.modificarPerfil	id existente	{exito:true, mensaje:'Perfil actualizado'}	Clase válida

REQ004 — Gestión de Pacientes

ID	Método / función interna	Entrada	Salida esperada	Técnica
UT-004-1	validarCedulaEcuatoriana (vía registrarPaciente)	'1710034065' (cédula válida real, prov. 17, dígito verificador correcto)	delega en servicioBase.registrarPaciente(..)	Clase válida
UT-004-2	validarCedulaEcuatoriana	'171003406' (9 dígitos)	{exito:false, mensaje:'La cédula ecuatoriana ingresada no es válida'}	Valor límite (longitud)
UT-004-3	validarCedulaEcuatoriana	'9910034065' (provincia 99, fuera de 1-24)	mensaje de cédula inválida	Clase inválida

UT-004-4	validarCedulaEcuatoriana	3er dígito = 6 (>5, no es persona natural)	mensaje de cédula inválida	Valor límite
UT-004-5	validarCedulaEcuatoriana	10 dígitos con dígito verificador alterado en 1	mensaje de cédula inválida	Clase inválida
UT-004-6	validarTextoSeguro (nombre)	nombre = "<script>alert(1)</script>"	{exito:false, mensaje:'El nombre es inválido o contiene contenido no permitido'}	Inyección/seguridad
UT-004-7	validarTextoSeguro (nombre)	nombre = " o solo espacios	mensaje de nombre inválido	Valor límite
UT-004-8	validarTelefono	'098765432' (9 dígitos)	{exito:false, mensaje:'El teléfono debe tener 10 dígitos'}	Valor límite
UT-004-9	validarCorreo	'usuario@dominio' (sin TLD)	{exito:false, mensaje:'El correo electrónico no es válido'}	Clase inválida
UT-004-10	validarFechaNacimiento	fecha = mañana	{exito:false, mensaje:'La fecha de nacimiento no es válida...'} {exito:true, mensaje:'La fecha de nacimiento es válida'}	Valor límite (futuro)
UT-004-11	validarFechaNacimiento	fecha = hoy - 120 años exactos	true (límite permitido)	Valor límite
UT-004-12	validarFechaNacimiento	fecha = hoy - 121 años	false (rechazada)	Valor límite
UT-004-13	ServicioPacientes.cambiarEstado	id existente, nuevoEstado='Inactivo'	{exito:true, mensaje:'Estado actualizado', paciente}	Clase válida
UT-004-14	ServicioPacientes.cambiarEstado	id inexistente	{exito:false, mensaje:'Paciente no encontrado'}	Clase inválida

REQ003 — Gestión de Citas

ID	Método	Entrada	Salida esperada	Técnica
UT-003-1	DisponibilidadDecorador.registrarCita	duracion = '0'	{exito:false, mensaje:'La duración de la cita no es válida'}	Valor límite
UT-003-2	DisponibilidadDecorador.registrarCita	duracion = 120	pasa la validación de duración (no rechaza)	Valor límite (permitido)

UT-0 03-3	DisponibilidadDecorador.registrarCita	duracion = 121	{exito:false, mensaje:'Una cita no puede durar más de 2 horas'}	Valor límite (rechazado)
UT-0 03-4	DisponibilidadDecorador.registrarCita	fecha/hora = ayer	{exito:false, mensaje:'No se puede agendar una cita en una fecha u hora pasada'}	Valor límite (tiempo, con fake timers)
UT-0 03-5	DisponibilidadDecorador.registrarCita	mismo consultorio+fecha, rango se solapa con cita existente activa	{exito:false, mensaje:'El consultorio ya tiene una cita agendada en ese horario'}	Clase inválida
UT-0 03-6	DisponibilidadDecorador.registrarCita	misma franja pero la cita existente está 'Cancelada'	delega en servicioBase.registrarCita (no cuenta como solapamiento)	Clase válida (excepción de estado)
UT-0 03-7	DisponibilidadDecorador.registrarCita	nueva cita inicia justo cuando termina la existente (fin==inicio)	no hay solapamiento (comparación estrictamente '<'), delega en servicioBase	Valor límite exacto
UT-0 03-8	DisponibilidadDecorador.reprogramarCita	id inexistente	{exito:false, mensaje:'Cita no encontrada'}	Clase inválida
UT-0 03-9	DisponibilidadDecorador.reprogramarCita	reprogramar a la misma franja de la propia cita (excluye su propio id)	no detecta autosolapamiento, delega en servicioBase	Exclusión por id (spy)
UT-0 03-10	ServicioCitas.registrarCita	datos válidos	citaRepo.guardar llamado; gestor.notificar con {tipo:'CITA_REGISTRADA'}; retorna {exito:true, cita}	Mock/spy
UT-0 03-11	ServicioCitas.cancelarCita	citaRepo.actualizar devuelve null	{exito:false, mensaje:'Cita no encontrada'}	Clase inválida
UT-0 03-12	ServicioCitas.cancelarCita	cita existente	gestor.notificar con {tipo:'CITA_CANCELADA'}; {exito:true}	Mock/spy

REQ006 — Gestión de Planificación

ID	Método	Entrada	Salida esperada	Técnica
UT-00 6-1	registrarHorario	falta horaFin (undefined)	{exito:false, mensaje:'Complete todos los datos del horario'}	Valor límite (campo faltante)
UT-00 6-2	registrarHorario	cupos = 0	{exito:false, mensaje:'El cupo mínimo es 1'}	Valor límite

UT-00 6-3	registrarHorario	cupo = 1, resto de datos completos	horario creado, horariosRepo.guardar llamado, se registra auditoría	Valor límite (permitido)
UT-00 6-4	modificarHorario	id inexistente	{exito:false, mensaje:'Horario no encontrado'}	Clase inválida
UT-00 6-5	modificarHorario	cambios.cupo = 0	{exito:false, mensaje:'El cupo mínimo es 1'}	Valor límite
UT-00 6-6	modificarHorario	citasAsignadas=5, cambios.cupo=4	{exito:false, mensaje:'No se puede reducir el cupo por debajo de las citas asignadas'}	Valor límite
UT-00 6-7	modificarHorario	citasAsignadas=5, cambios.cupo=5	modificación aceptada (límite exacto)	Valor límite (permitido)
UT-00 6-8	eliminarHorario	horario con citasAsignadas=1	{exito:false, mensaje:'No se puede eliminar un horario con citas dependientes'}	Regla de negocio
UT-00 6-9	inactivarHorario	horario con citasAsignadas=0	estado pasa a 'Inactivo', horariosRepo.actualizar llamado	Clase válida
UT-00 6-10	eliminarConsultorio	consultorio con ≥ 1 horario 'Activo' asociado	{exito:false, mensaje:'No se puede eliminar un consultorio con horarios activos'}	Regla de negocio
UT-00 6-11	asignarFisioterapeutaYConsultorio	id existente	horario actualizado con fisioterapeuta/consultorioid; se registra auditoría	Clase válida

REQ002 — Generar Notificación

Clases reales: NotificacionObserver (orquestración + mensaje) → ServicioNotificaciones (persistencia del envío). Requiere jsdom para document.getElementById.

ID	Método	Entrada	Salida esperada	Técnica
UT-002-1	NotificacionObserver.actualizar	evento.tipo = 'LOGIN'	no llama procesarEventoCita (solo procesa tipos 'CITA_*')	Filtro de eventos
UT-002-2	NotificacionObserver.generarMensaje	tipo='CITA_REGISTRADA', cita completa	"Su cita <id> ha sido registró..." con fecha/hora/servicio/estado reales	Clase válida
UT-002-3	NotificacionObserver.generarMensaje	cita sin fecha/hora/servicio/estado	usa valores por defecto: 'N/A', 'N/A', 'Servicio no definido', 'Desconocido'	Valor límite (campos faltantes)

UT-002-4	NotificacionObserver.procesarEventoCita	paciente no encontrado (pacienteRepo.obtenerPorId → undefined)	mostrarAviso('No se encontró el paciente...', 'error'); no se registra notificación	Clase inválida
UT-002-5	NotificacionObserver.procesarEventoCita	paciente sin correo ni teléfono, canalPreferido='Correo'	servicioNotificaciones.registrarPendiente llamado con motivo 'Faltan datos de contacto para Correo'	Clase inválida
UT-002-6	NotificacionObserver.procesarEventoCita	paciente con canalPreferido='WhatsApp' y teléfono válido	destinatario = teléfono; servicioNotificaciones.enviar llamado con canal 'WhatsApp'	Clase válida
UT-002-7	ServicioNotificaciones.enviar	datos completos	notificacionRepo.guardar llamado con estado:'Enviado'; gestor.notificar con {tipo:'NOTIFICACION_REGISTRADA'}	Mock/s py
UT-002-8	ServicioNotificaciones.registrarPendiente	motivo dado	registro guardado con estado:'Pendiente'; retorna {exito:false, mensaje:motivo}	Clase inválida
UT-002-9	GestorEventos.notificar	2 observadores suscritos	ambos observadores reciben actualizar(evento) exactamente 1 vez	Mock/s py

7. Matriz de Responsabilidades (RACI)

Requisito	Desarrollador	Diseño de casos	Ejecución	Revisión QA
REQ000 — Login	Saray Cañarte	Saray Cañarte	Doménica Villagómez	Danny Ayuquina
REQ001 — Recuperación	Doménica Villagómez	Doménica Villagómez	Saray Cañarte	Danny Ayuquina
REQ005 — Perfiles	Danny Ayuquina	Danny Ayuquina	Saray Cañarte	Doménica Villagómez
REQ004 — Pacientes	Doménica Villagómez	Doménica Villagómez	Danny Ayuquina	Saray Cañarte
REQ003 — Citas	Saray Cañarte	Saray Cañarte	Doménica Villagómez	Danny Ayuquina
REQ006 — Planificación	Saray Cañarte	Saray Cañarte	Danny Ayuquina	Doménica Villagómez
REQ002 — Notificaciones	Danny Ayuquina	Danny Ayuquina	Saray Cañarte	Doménica Villagómez